Portfolio 2: Query Processing and Optimisation

Project: Stretto Technologies Database System.

Database Size: 50,000 Transactional Records.

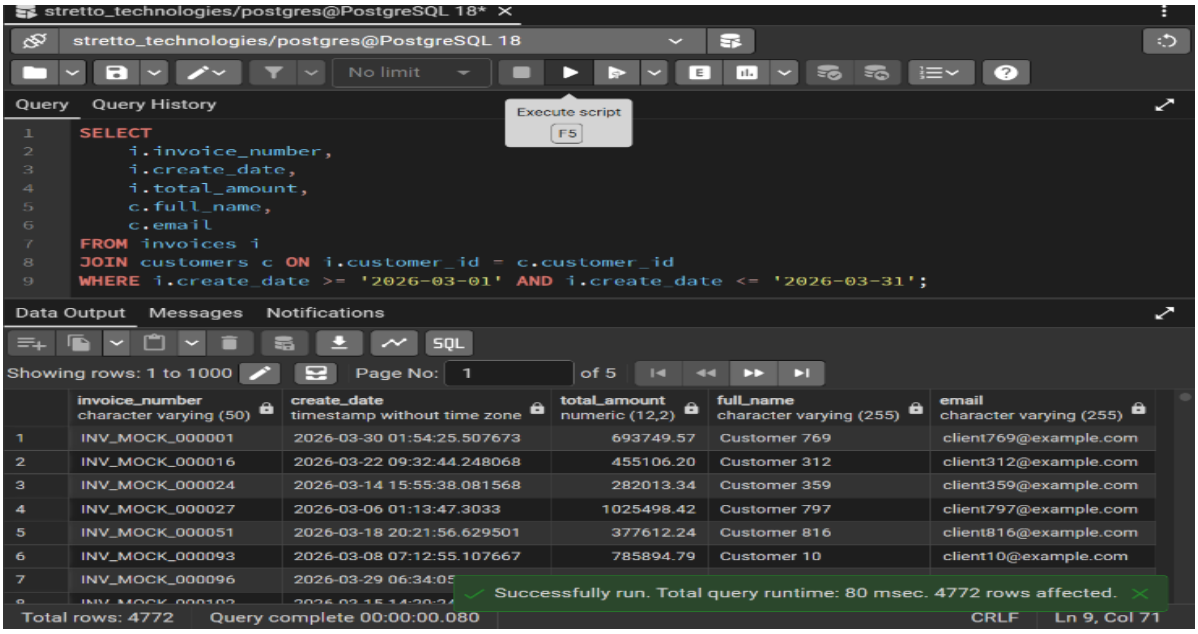
1. Business Analytical Queries

To demonstrate complex data retrieval, three analytical queries were developed to extract operational intelligence from the Stretto Technologies database.

A. Date-Range Transaction Filtering :This query uses an INNER JOIN to retrieve customer contact details alongside their invoices for March 2026.

SQL :

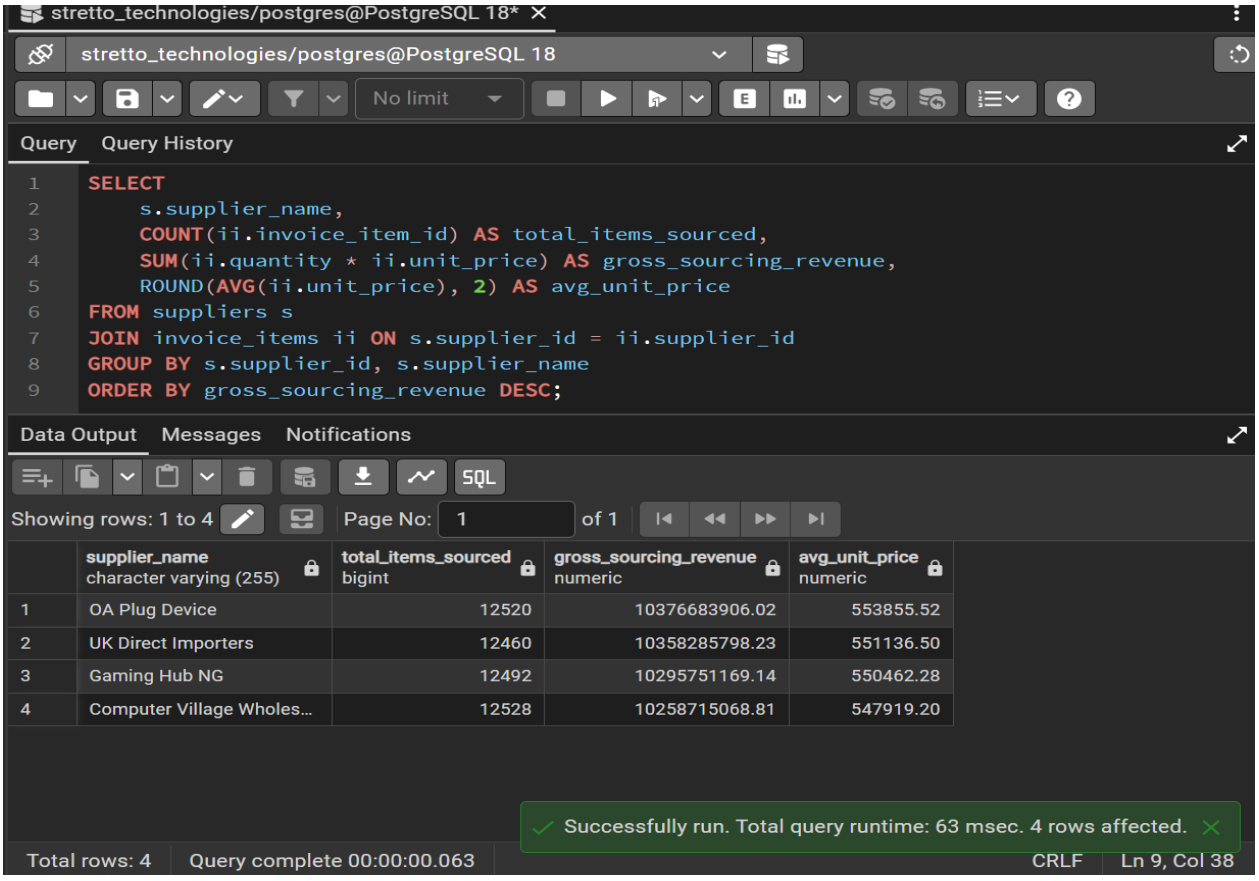
```
SELECT
    i.invoice_number,
    i.create_date,
    i.total_amount,
    c.full_name,
    c.email
FROM invoices i
JOIN customers c ON i.customer_id = c.customer_id
WHERE i.create_date >= '2026-03-01' AND i. create_date <= '2026-03-31';
```



B. <u>Supplier Sourcing Revenue Aggregation</u>: This query calculates total items sourced and gross revenue generated per supplier using the SUM(), COUNT(), and GROUP BY functions.

SQL:

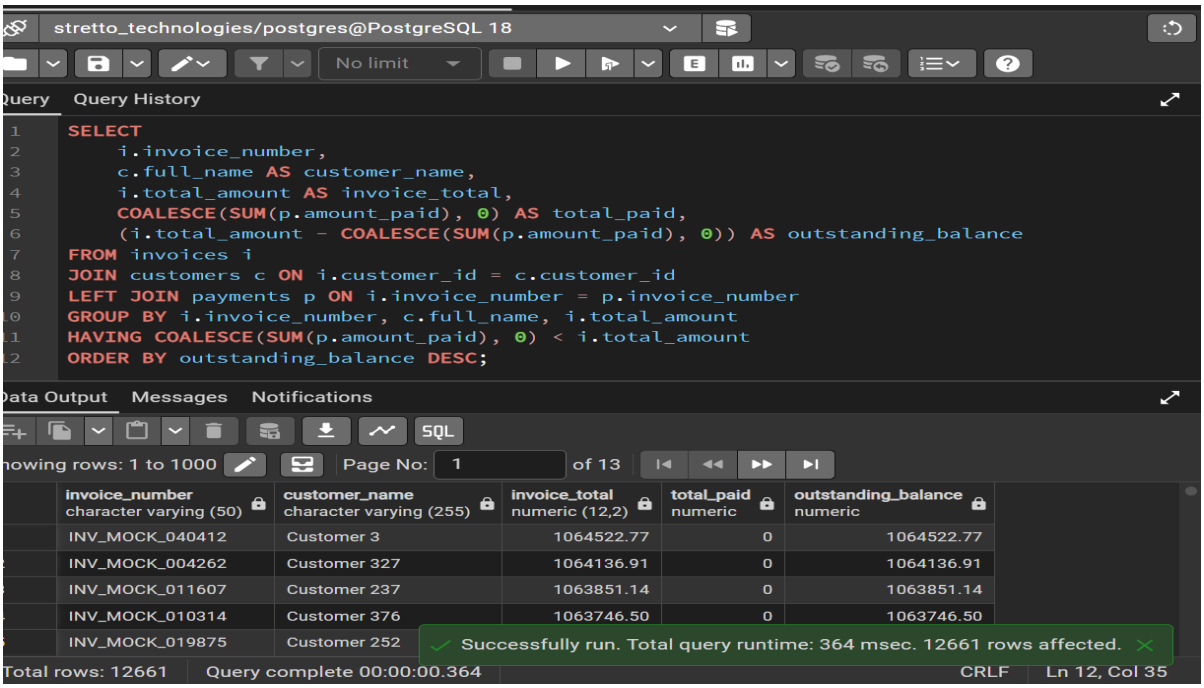
```
SELECT
    s.supplier_name,
    COUNT(ii.invoice_item_id) AS total_items_sourced,
    SUM(ii.quantity * ii.unit_price) AS gross_sourcing_revenue,
    ROUND(AVG(ii.unit_price), 2) AS avg_unit_price
FROM suppliers s
JOIN invoice_items ii ON s.supplier_id = ii.supplier_id
GROUP BY s.supplier_id, s.supplier_name
ORDER BY gross_sourcing_revenue DESC;
```



C. <u>Outstanding Balance Reconciliation:</u> This query identifies customers with unpaid or partially paid invoices. It uses a LEFT JOIN on the payments table and the COALESCE function to handle missing payment records, filtering for outstanding balances using the HAVING clause.

SQL:

```
SELECT
    i.invoice_number,
    c.full_name AS customer_name,
    i.total_amount AS invoice_total,
    COALESCE(SUM(p.amount_paid), 0) AS total_paid,
    (i.total_amount - COALESCE(SUM(p.amount_paid), 0)) AS outstanding_balance
FROM invoices i
JOIN customers c ON i.customer_id = c.customer_id
LEFT JOIN payments p ON i.invoice_number = p.invoice_number
GROUP BY i.invoice_number, c.full_name, i.total_amount
HAVING COALESCE(SUM(p.amount_paid), 0) < i.total_amount
ORDER BY outstanding_balance DESC;
```



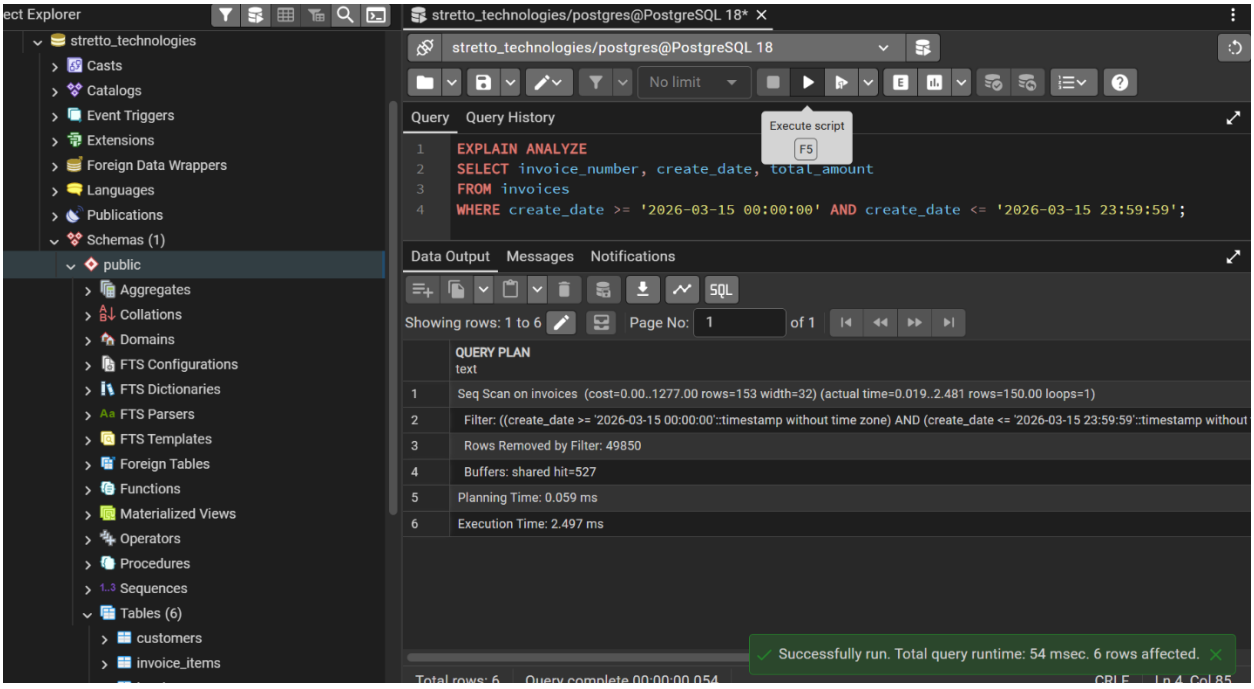
2. Query Performance & Execution Plan Tuning

To demonstrate query optimization on a large dataset (50,000 rows), a baseline execution plan was captured for a date-range filter, followed by the implementation of a B-Tree index to improve retrieval speeds.

A. **Baseline Execution (Unoptimized)**: Before indexing, the EXPLAIN ANALYZE command was run on the date-range query. The database engine was forced to perform a **Sequential Scan (Seq Scan)**, reading all 50,000 rows line-by-line, resulting in an execution time of approximately 54 milliseconds.

SQL:

```
EXPLAIN ANALYZE
SELECT invoice_number, create_date, total_amount
FROM invoices
WHERE create_date >= '2026-03-15 00:00:00' AND create_date <= '2026-03-15
23:59:59';
```



B. **<u>Index Implementation</u>**: To optimize the retrieval path, an index was created on the create_date column, and table statistics were updated.

SQL:
CREATE INDEX idx_invoices_create_date ON invoices(create_date);
ANALYZE invoices;

C. **<u>Optimized Execution (Post-Index):</u>** When the exact same EXPLAIN ANALYZE query was executed for a specific day, the execution plan successfully transitioned to a **Bitmap Heap Scan**. The database utilized the index to pinpoint the exact memory pages, bypassing the sequential scan and dropping the execution time to about 38 milliseconds.

SQL:

EXPLAIN ANALYZE
SELECT invoice_number, create_date, total_amount
FROM invoices
WHERE create_date >= '2026-03-15 00:00:00' AND create_date <= '2026-03-15 23:59:59';

